Twinfield web services – Pay and collect

manual, version 5.4

Oktober 2010

Twinfield International NV
De Wel 20-b
3871 MV Hoevelaken
Netherlands

the no. 1 in online accounting

Table of contents

Table of contents 2
Payment or collect cycle 3
Resources 3
ProcesXmlString 3
General 4
Classes 5
Methods 10
Mappings 24

The paylist 26

Enumerations 26
Classes 26
Methods 27

the no. 1 in online accounting

Payment or collect cycle

This chapter is a functional description of the Twinfield Process Xml web service, which can be used to exchange information with Twinfield.

The Twinfield payment and collect web service contains the following functions:

- Authorize Authorizes a payment run
- CreateFile Creates a pay file from a pay run.
- CreateRun Creates a new pay run in an office.
- Finish Closes a pay run.
- GetRunInfo Reads detailed data of a single pay run.
- GetRuns Reads the existing pay runs from one office.
- Modify Changes payment values of an existing pay run.
- Post Posts payments of a pay run to the 'in-transfer' ledger.

Resources

resource	location
Web Service	/webservices/payandcollect.asmx
WSDL File	/webservices/payandcollect.asmx?wsdl

ProcesXmlString

```
public string ProcessXmlString(
     string xmlRequest
)
```

Methods

The sessionID must be sent as a SOAP header.

Parameters

parameter	description
xmlRequest	Xml request string.

Return value

Xml result string.

the no. 1 in online accounting

General

Generating a pay file from Twinfield is done in a number of steps.

1. You first need to generate a proposal of payments.
2. This proposal can be manipulated: payments and complete dimensions can be excluded, values of payments can be altered.
3. If manipulating is done, the run can be authorized. After authorizing, the run data cannot be altered again.
4. If the run is authorised, the payments can be posted to the 'in transfer' ledger, and will be matched against the original invoices.
5. After posting, the payment file can be generated.

Enumerations

```
public enum RunType
{
   Payment
   DirectDebit
}

public enum Method
{
   Detailed
   Summarise
   SummariseAndDeduct
}

public enum Phase
{
   Select
   Manipulate
   Authorise
   Post
   Pay
   Finalise
}

public enum PhaseStatus
{
   Started
   Done
   DoneWithErrors
}

public enum MessageType
{
   Error
   Warning
   Informational
}
```


the no. 1 in online accounting

Classes

PayRun

```
public Class PayRun
{
   public string Office;
   public string PayType;
   public int Id;
   public DateTime Created;
   public DateTime Modified;
   public int Touched;
   public string User;
   public RunType RunType;
   public Phase Phase;
   public PhaseStatus Status;
   public string Bank;
   public DateTime PayDate;
   public int Year;
   public int Period;
   public bool DirectRemove;
   public string[][] CustomSettings;
   public SummaryInfo[] Summaries;
}
```

Parameters

parameter	description
Office	The office where the pay run belongs to
PayType	The code of the pay type
Id	The unique id of the pay run
Created	The creation date of this run
Modified	The date and time the run data was last modified
Touched	The number of times the run data was modified
User	The user who made the last modifications
RunType	The type of the run
Phase	The current phase of the pay run
Status	The current status of the phase

the no. 1 in online accounting

parameter	description
Bank	The code of the bank that's associated with this pay run
PayDate	The pay date of the payment file
Year	The year where the transactions will be booked on
Period	The period the transactions will be booked on
DirectRemove	The direct Remove setting for this run
CustomSettings	A string array of string arrays with custom pay type settings
Summaries	An array of SummaryInfo Objects

Example

```
// Build customSettings array.
   //BTL91 has settings to determine who should pay the transfer costs.
   string[][] CustomSettings = new string[2][];
   CustomSettings[0] = new string[] { "PayCostNational", "0" };
   CustomSettings[1] = new string[] { "PayCostInternational", "0" };
```

Summary

```
   public class SummaryInfo
   {
      public string Relation;
      public string RelationName;
      public Bank RelationBank;
      public string Currency;
      public string CurrencyName;
      public int Id;
      public string Reference;
      public bool Included;
      public DetailInfo[] Details;
   }
```

parameter	description
Relation	Code of the relation for this summary
RelationName	Name of the relation for this summary
RelationBank	An instance of the Bank object, containing bank details of the relation
Currency	Currency of this summary

the no. 1 in online accounting

parameter	description
CurrencyName	Name of the currency this summary is in
Id	Identifier
Reference	Pay reference, formed as set in the pay file settings
Included	Includes/excludes a summary from this pay run
Details	An array of DetailInfo objects

DetailInfo

```
public class DetailInfo
{
   public int Id;
   public string Transaction;
   public string TransactionName;
   public int TransactionNumber;
   public int TransactionLine;
   public string InvoiceNumber;
   public DateTime InvoiceDate;
   public DateTime InvoiceDueDate;
   public Decimal Value;
   public Decimal OriginalValue;
   public Decimal BaseValue;
   public Decimal OriginalBaseValue;
   public bool Included;
}
```

Explanation

parameter	description
Id	Unique identifier of the summary detail
Transaction	Code of the invoice transaction
TransactionName	Name of the invoice transaction
TransactionNumber	Number of the invoice transaction
TransactionLine	Line number of the invoice transaction
InvoiceNumber	The original invoice number of the transaction

the no. 1 in online accounting

parameter	description
InvoiceDate	The date of the invoice
InvoiceDueDate	The due date of the invoice
Value	the pay value of the detail
OriginalValue	Original, not manipulated value of the summary detail in the summaries currency
BaseValue	Value of the summary detail in the base currency
OriginalBaseValue	Original, not manipulated value of the summary detail in the base currency
Included	Includes/excludes a detail from this summary

SummarySettings

```
public class SummarySettings
{
    public string Relation;
    public string Currency;
    public int Id;
    public bool Included;
    public DetailSettings[] Details;
}
```

properties

Relation, Currency and Id form the key of a summary.

parameter	description
Relation	The code of this summary's relation
Currency	The currency of this summary
Id	Identifier
Included	Includes/excludes a summary from this pay run
Details	Modifiable details of this summary

the no. 1 in online accounting

DetailSettings

```
public class DetailSettings
{
   public int Id;
   public Decimal Value;
   public bool Included;
}
```

Properties

property	description
Id	Unique identifier of the summary detail
Value	The pay value of the detail
Included	Includes/excludes a detail from this summary

Bank

```
public class Bank
{
      public string Ascription;
      public string AccountNumber;
      public string Iban;
      public string NationalCode;
      public string InternationalCode;
      public string Address;
      public string PostalCode;
      public string Branch;
      public string Country;
}
```

Properties

property	description
Ascription	The name of the account holder
AccountNumber	The number of the account
Iban	The international number of the account
NationalCode	The national identifying code of the bank
InternationalCode	The international identifying code of the bank
Address	The street name and number of the bank
PostalCode	The postal code of the address
Branch	The city where the bank is located

the no. 1 in online accounting

property	description
Country	The country where the bank is located

Methods

With every step, thou shall send thy sessionID as a SOAP header.

GetRunInfo

To generate details of a particular single run, the following function is available:

```
public PayRun GetRunInfo(
   string office,
   string payType,
   int runId)
```

Parameters

parameter	description
office	The office from which the current pay runs should be selected
payType	The code of the single run that should be selected
runId	The run Id of the single run that should be selected

The parameters will be parsed to an xml document:

```
<read>
   <type>paylist</type>
   <office>string</office>
   <code>string</code>
   <runid>int</runid>
</read>
```

The above xml will be send to ProcessXML.
The core will return the following xml document:

```
<payrun>
   <office name="string" shortname="string">string</office>
   <code name="string" shortname="string">string</code>
   <type>RunType</type>
   <number>int</number>
   <user>string</user>
   <status>RunStatus</status>
   <phase>Phase</phase>
   <warning>int</warning>
   <bank name="string" shortname="string">string</bank>
   <wrdcode>string</wrdcode>
```


the no. 1 in online accounting

```
   <report>string</report>
   <paydate>DateTime</paydate>
   <year>int</year>
   <period>int</period>
   <basecurrency>string</basecurrency>
   <origbasevalue>Decimal</origbasevalue>
   <basevalue>Decimal</basevalue>
   <started>DateTime</started>
   <modified>DateTime</modified>
   <remove>bool</remove>
   <touched>int</touched>
   <summaries first="int" last="int" total="int">
      <summary>
         <dimension name="string" shortname="string">string</dimension>
         <currency name="string" shortname="string">string</currency>
         <sumid>int</sumid>
         <value>decimal</value>
         <origvalue>decimal</origvalue>
         <basecurrency>string</basecurrency>
         <basevalue>decimal</basevalue>
         <origbasevalue>decimal</origbasevalue>
         <deleted>boolean</deleted>
         <status>int</status>
         <trscode name="string" shortname="string">string</trscode>
         <trsnumber>int</trsnumber>
         <payreference>string</payreference>
         <details first="int" last="int" total="int">
            <detail>
               <id>int</id>
               <trscode name="string" shortname="string">string</trscode>
               <trsnumber>int</trsnumber>
               <trsline>int</trsline>
               <value>decimal</value>
               <origvalue>decimal</origvalue>
               <basevalue>decimal</basevalue>
               <origbasevalue>decimal</origbasevalue>
               <deleted>boolean</deleted>
               <freetext1>string</freetext1>
               <freetext2>string</freetext2>
               <freetext3>string</freetext3>
               <invnumber>string</invnumber>
               <description>string</description>
               <rate>decimal</rate>
            </detail>
         </details>
      </summary>
   </summaries>
</payrun>
```

CreateRun

Creating a proposal consists of two steps:
You must first create a unique run identifier.
After that, you can generate the data.
This means that the CreateRun() function makes 2 calls to ProcessXML.

the no. 1 in online accounting

```
public enum CreateRunResult
{
   UnknownTextualMessage
}

public class MessageOfCreateRunResult
{
   public MessageType Type;
   public string Text;
   public CreateRunResult Code;
   public string[] Parameters;
}

   public MessageOfCreateRunResult[] CreateRun(
      string office,
      string payType,
      DateTime payDate,
      string period,
      Method method,
      bool directRemove,
      string bankCode,
      string relationFrom,
      string relationTo,
      DateTime maxDuedate,
      string invoiceNumber,
      string currency,
      string[][] customSettings,
      out PayRun payRun)
```

Parameters

parameter	Mandatory	description
office	Yes	The office the run should be created in
payType	Yes	The pay type of which the run must be created
payDate	Yes	The date the payments must be payed
period	Yes	The period the payment transactions must be booked on e.g. 2008/01
method *	No	The summarisation method that should be used for this run
directRemove *	No	Determines weather the excluded payments should be available after authorising this run
bankCode *	No	The bank this pay file will be send to
relationFrom	No	Filter on level 2 dimension
relationTo	No	Filter on level 2 dimension
maxDueDate	No	Filter on invoices maximum due date

the no. 1 in online accounting

parameter	Mandatory	description
invoiceNumber	No	Filter on invoice number
currency	No	Filter on invoice currency
customSettings *	No	String array of string arrays with custom settings, depending on the pay type

* If the values differ from the pay type settings, and the prompt values of the default pay type settings are set to false, an error message will be returned.

Creating a unique run is called the 'SelectStart'.
It creates an entry in the pay data, and locks this entry, so no alterations can be made until the selection is complete.
The xml document that is needed to do this looks like this:

```
<payrun action="selectstart">
   <office>string</office>
   <code>string</code>
   <paydate>DateTime</paydate>
   <period>string</period>
   <bank>string</bank>
   <remove>string</remove>
   <customsetting1>string</customsetting1>
   <customsetting2>string</customsetting2>
</payrun>
```

Node	explanation
payrun	@action defines the run phase.
office	The office the run will be created in.
code	The pay type which will be used.
paydate	the date the payments should be executed.
period	The period the payments should be posted on.
bank [1]	This is the bank where the payments shall be send to.
remove [1]	Determines if excluded invoices/dimensions should be removed from the pay run while posting. this will make these invoices/dimensions available for a new run after posting, otherwise, they will be available again after finishing the complete run.
customsetting 1 to n [1]	settings that can differ per pay type

the no. 1 in online accounting

[1] These nodes are not mandatory, and will be generated from pay type management if missing.

If successful, the returned xml document should look like this:

```
<payrun action="selectstart" result="1">
   <office>string</office>
   <code>string</code>
   <touched>int</touched>
   <number>int</number>
</payrun>
```

After 'selectstart', the real selection can take place.

To start the selecting procedure, the following xml document should be send to Twinfield:

```
<payrun action="select">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <method>int</method>
   <prompts>
      <prompt>
         <id>int</id>
         <from>string</from>
         <to>string</to>
      </prompt>
      <prompt>
         <id>int</id>
         <from>string</from>
      </prompt>
      <prompt>
         <id>int</id>
         <from>string</from>
      </prompt>
      <prompt>
         <id>int</id>
         <from>string</from>
      </prompt>
   </prompts>
</payrun>
```


the no. 1 in online accounting

Used selection methods:
There are three ways of selecting data.
If a run is a pay run, all associated Purchase Invoices can be selected.
If a run is a direct debit run, all associated Sales Invoices can be selected.
The method node is not mandatory, and will be generated by the pay type management if missing.

method	description
0	detailed: The invoices will not be summarised
1	summarize: All invoices will be summarizes per dimension/currency, no credit invoices will be selected
2	Summarize and deduct: all invoices will be selected, including credit invoices, and summarized/deducted per dimension/currency

Prompts:
There are a few standard filters you can set before selecting:
A range of dimensions(accounts payable or accounts receivable)(prompt id = 3)
A particular Invoice. (prompt id = 5)
A maximum due date. (prompt id = 6)
In some occasions (if a pay type supports multiple currencies) the currency(prompt id = 7).
If the selection succeeded, the xml document that is returned contains all relevant run data:

Payrun.xml

```
<payrun>
   <office name="string" shortname="string">string</office>
   <code name="string" shortname="string">string</code>
   <type>int</type>
   <number>int</number>
   <user>string</user>
   <status>int</status>
   <phase>int</phase>
   <warning>int</warning>
   <bank name="string" shortname="string">string</bank>
   <wrdcode>string</wrdcode>
   <report>string</report>
   <paydate>DateTime</paydate>
   <year>int</year>
   <period>int</period>
   <basecurrency>string</basecurrency>
   <origbasevalue>Decimal</origbasevalue>
   <basevalue>Decimal</basevalue>
   <started>DateTime</started>
   <modified>DateTime</modified>
   <remove>bool</remove>
   <touched>int</touched>
   <summaries first="int" last="int" total="int">
      <summary>
         <dimension name="string" shortname="string">string</dimension>
         <currency name="string" shortname="string">string</currency>
```


the no. 1 in online accounting

```
            <sumid>int</sumid>
            <value>decimal</value>
            <origvalue>decimal</origvalue>
            <basecurrency>string</basecurrency>
            <basevalue>decimal</basevalue>
            <origbasevalue>decimal</origbasevalue>
            <deleted>boolean</deleted>
            <status>int</status>
            <trscode name="string" shortname="string">string</trscode>
            <trsnumber>int</trsnumber>
            <payreference>string</payreference>
            <details first="int" last="int" total="int">
               <detail>
                  <id>int</id>
                  <trscode name="string" shortname="string">string</trscode>
                  <trsnumber>int</trsnumber>
                  <trsline>int</trsline>
                  <value>decimal</value>
                  <origvalue>decimal</origvalue>
                  <basevalue>decimal</basevalue>
                  <origbasevalue>decimal</origbasevalue>
                  <deleted>boolean</deleted>
                  <freetext1>string</freetext1>
                  <freetext2>string</freetext2>
                  <freetext3>string</freetext3>
                  <invnumber>string</invnumber>
                  <description>string</description>
                  <rate>decimal</rate>
               </detail>
            </details>
         </summary>
      </summaries>
      <selectedtransactions>int</selectedtransactions>
      <skippedtransactions>int</skippedtransactions>
</payrun>
```

This xml document shall be parsed to the PayRun Class and returned to the caller.
Eventual errors, the count of selected items, the count of rejected items, and the reasons for rejection will be returned in the ResultMessages object.

Modify

Modify is used to exclude dimensions or invoices from the run data, or alter amounts of invoices.
We have chosen to use a 'light' summary object, stripped to identifiers and writeable values only, to reduce the data that will be send over the net.
Conditions:

1. Phase = Select and Status = Done and selected Transactions > 0
2. Phase = Manipulate and Status <> Busy

```
public enum ModifyResult
{
   UnknownTextualMessage
```

```
}

public class MessageOfModifyResult
{
   public MessageType Type;
   public string Text;
   public ModifyResult Code;
   public string[] Parameters;
}

   public MessageOfModifyResult[] Modify(
      string office,
      string payType,
      int runId,
      ref int touched,
      ref DateTime modified,
      SummarySettings[] newSummaries)
```

parameters	description
office	The office from which the run should be modified
payType	The code of the run that should be modified
runId	The run Id of the run that should be modified
touched	The touched figure, retrieved from the last request to this particular run
modified	The modified date and time, retrieved from the last request to this particular run

Since the function only provides the SummarySettings, and the core expects a complete run object, we'll have to make a call to the core to get the complete PayRun object for this run, and merge the altered SummarySettings into the SummaryInfo.
With that we create the following xml:

```
<payrun action="manipulate">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <modified>string</modified>
   <summaries>
      <summary>
         <dimension>string</dimension>
         <currency>string</currency>
         <sumid>int</sumid>
         <value>decimal</value>
         <origvalue>decimal</origvalue>
         <basecurrency>string</basecurrency>
         <basevalue>decimal</basevalue>
         <origbasevalue>decimal</origbasevalue>
         <deleted>boolean</deleted>
```


the no. 1 in online accounting

```
            <status>int</status>
            <trscode>string</trscode>
            <trsnumber>int</trsnumber>
            <payreference>string</payreference>
            <details>
               <detail>
                  <id>int</id>
                  <trscode name="string" shortname="string">string</trscode>
                  <trsnumber>int</trsnumber>
                  <trsline>int</trsline>
                  <value>decimal</value>
                  <origvalue>decimal</origvalue>
                  <basevalue>decimal</basevalue>
                  <origbasevalue>decimal</origbasevalue>
                  <deleted>boolean</deleted>
                  <freetext1>string</freetext1>
                  <freetext2>string</freetext2>
                  <freetext3>string</freetext3>
                  <invnumber>string</invnumber>
                  <description>string</description>
                  <rate>decimal</rate>
               </detail>
            </details>
         </summary>
      </summaries>
   </payrun>
```

This xml document will be send to ProcessXml.
ProcessXml will return the same xml document, with eventual messages on the nodes where things went wrong.
These messages will be returned in an array of ResultMessage objects.
When manipulating, only alterations of the following values will be accepted:

Field	**values**
PayRun.Summaries[x].Included	True or False
PayRun.Summaries[x].Details[y].Included	True or False
PayRun.Summaries[x].Details[y].Value	Unequal to zero, can not be greater than PayRun.Summaries[x].Details[y].OriginalValue

the no. 1 in online accounting

(a) Authorise

Authorise is used to freeze the run, after authorisation, no adjustments can be made to the data.
Conditions:

1. Phase <= Manipulate and Status = Done

```
public enum AuthoriseResult
{
   UnknownTextualMessage
}

public class MessageOfAuthoriseResult
{
   public MessageType Type;
   public string Text;
   public AuthoriseResult Code;
   public string[] Parameters;
}

   public MessageOfAuthoriseResult[] Authorize(
      string office,
      string payType,
      int runId,
      ref int touched,
      ref DateTime modified)
```

These parameters will be parsed to the following xml document:

```
<payrun action="authorize">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <modified>string</modified>
</payrun>
```

On success, the following xml document is returned:

```
<payrun action="authorize" result="1">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <modified>string</modified>
</payrun>
```

Eventual errors will be returned in an array of the ResultMessage object.

(b) Post

Post will post the summarisations to the 'in-transfer' ledger, and match them against the original transactions.
If post fails, alterations can be made in Twinfield, and the post process can be started again.
To prevent postings to be started twice, the pay run must be locked before we can start the postings.
This means that Post() will make 2 calls to ProcessXml.

the no. 1 in online accounting

Conditions:

1. Phase = Authorise and Status = Done
2. Phase = Post and Status = DoneWithErrors

```
public enum PostResult
{
   UnknownTextualMessage
}

public class MessageOfPostResult
{
   public MessageType Type;
   public string Text;
   public PostResult Code;
   public string[] Parameters;
}

public MessageOfPostResult[] Post(
   string office,
   string payType,
   int runId,
   ref int touched,
   ref DateTime modified)
```

To lock the pay run, the following xml document should be send to Twinfield:

```
<payrun action="poststart">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <modified>string</modified>
</payrun>
```

On success, this will result in the following xml document:

```
<payrun action="poststart" result="1">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <modified>string</modified>
</payrun>
```

On success, the actual posting can take place, this is started by sending the following xml document to Twinfield:

```
<payrun action="post">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <modified>string</modified>
</payrun>
```

On success, this will result in the following xml document:

```
<payrun action="post" result="1">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <modified>string</modified>
```


the no. 1 in online accounting

```
</payrun>
```

Eventual errors will be parsed to an array of ResultMessage objects and returned.

(c) CreateFile

Conditions:

1. Phase >= Post and < Finalise and Status = Done

```
public enum CreateFileResult
{
   UnknownTextualMessage
}

public class MessageOfCreateFileResult
{
   public MessageType Type;
   public string Text;
   public CreateFileResult Code;
   public string[] Parameters;
}

   public MessageOfCreateFileResult[] CreateFile(
      string office,
      string payType,
      int runId,
      ref int touched,
      ref DateTime modified,
      string[][] exportSettings,
      out byte[] payFile)
```

parameter	description
office	The office from which the run should be authorised
payType	The code of the run that should be authorised
runId	The runId of the run that should be authorised
touched	The touched figure, retrieved from the last request to this particular run
modified	The modified date and time, retrieved from the last request to this particular run
exportSettings	An array of string arrays with optional exportsettings

Example

```
// Build exportSettings array.
//ClieOp03 has settings to determine if a file is a test file or
production file.
```


the no. 1 in online accounting

```
//ClieOp03 has settings to determine if a file is a original file or
duplicate file.
   string[][] exportSettings = new string[2][];
   exportSettings[0] = new string[] { "FileType", "production" };
   exportSettings[1] = new string[] { "Origin", "original" };
```

This step will export the file from the office in the xml, to the office you are logged on to, which is not by default the office in the xml document. After generating, the code will expand the returned XML with the FileId of the payment file.
The webservices will load the file, return the file as an array of byte, and delete the payment file from the filemanager.

Generating also consists of 2 steps:
Locking the payrun, and creating the file.
This means that CreateFile() will also make two calls to ProcessXml.
To lock the payrun, the following xml document should be send to Twinfield.

```
<payrun action="payfilestart">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <modified>string</modified>
</payrun>
```

On succes, this will result in the following xml document:

```
<payrun action="payfilestart" result="1">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <modified>string</modified>
</payrun>
```

To generate the payfile, the following xml document should be send to Twinfield:

```
<payrun action="payfilecreate">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <modified>string</modified>
   <exportsetting1>string</exportsetting1>
   <exportsetting2>string</exportsetting2>
   <exportsetting3>string</exportsetting3>
   <exportsetting4>string</exportsetting4>
</payrun>
```

On succes, this will result in the following xml document:

```
<payrun action="payfilecreate" result="1">
   <office>string</office>
   <code>string</code>
   <number>int</number>
```

```
   <touched>int</touched>
   <modified>string</modified>
   <fileid>int</fileid>
</payrun>
```

Eventually errors will be parsed to an array of ReturnMessage objects and returned.

Finish

Finishing a run can be used for 2 different things:

1. deleting a run, this can only be done if no postings to the 'in-transfer' ledger have taken place yet.
2. ending a run after generating the payfile.
   It deletes all payrundata from Twinfield.

Conditions:

1. Phase = Pay and Status = Done
2. Phase < Posting
3. Phase = Posting and Status = Error and no lines have been posted yet.

```
public enum FinishResult
{
   UnknownTextualMessage
}

public class MessageOfFinishResult
{
   public MessageType Type;
   public string Text;
   public FinishResult Code;
   public string[] Parameters;
}

public MessageOfFinishResult[] Finish(
   string office,
   string payType,
   int runId,
   int touched,
   datetime modified)
```

The following xml document should be created:

```
<payrun action="finish">
   <office>string</office>
   <code>string</code>
   <number>int</number>
   <touched>int</touched>
   <modified>string</modified>
</payrun>
```

This will delete all rundata from the database.

If errors occur, they will be parsed to an array of ReturnMessage objects and returned.

the no. 1 in online accounting

Mappings

class	field	Xpath
PayRun	string Office	payrun/office
PayRun	string Code	payrun/code
Payrun	int Id	payrun/number
PayRun	RunType Type	payrun/type
PayRun	Phase Phase	payrun/status
PayRun	PhaseStatus Status	payrun/phase
PayRun	string BankCode	payrun/bnkcode
PayRun	string WordTemplate	payrun/wrdcode
PayRun	DateTime PayDate	payrun/paydate
PayRun	int Year	payrun/year
PayRun	int Period	payrun/period
PayRun	DateTime StartDate	payrun/started
PayRun	int Modified	payrun/modified
PayRun	string LastUser	payrun/user
PayRun	int Touched	payrun/touched
PayRun	bool DirectRemove	payrun/remove
PayRun	string[][] CustomSettings	payrun/string[x][]
PayRun	SummaryInfo[] Summaries	payrun/summaries
SummaryInfo	string Office	payrun/office
SummaryInfo	string Code	payrun/code
SummaryInfo	int RunId	payrun/number
SummaryInfo	int Id	payrun/summaries/summary/sumid
SummaryInfo	string DimensionCode	payrun/summaries/summary/dimension
SummaryInfo	string DimensionName	payrun/summaries/summary/dimension/@name
SummaryInfo	string CurrencyCode	payrun/summaries/summary/currency

the no. 1 in online accounting

class	field	Xpath
SummaryInfo	string CurrencyName	payrun/summaries/summary/currency/@name
SummaryInfo	bool Included	payrun/summaries/summary/deleted
SummaryInfo	string Reference	payrun/summaries/summary/payreference
SummaryInfo	DetailInfo[] Details	payrun/summaries/summary/details
DetailInfo	string Office	payrun/office
DetailInfo	string Code	payrun/code
DetailInfo	int RunId	payrun/number
DetailInfo	int SumId	payrun/summaries/summary/sumid
DetailInfo	int Id	payrun/summaries/summary/details/detail/id
DetailInfo	string TransactionCode	payrun/summaries/summary/details/detail/trscode
DetailInfo	string TransactionName	payrun/summaries/summary/details/detail/trscode/@name
DetailInfo	int TransactionNumber	payrun/summaries/summary/details/detail/trsnumber
DetailInfo	int TransactionLine	payrun/summaries/summary/details/detail/trnline
DetailInfo	string InvoiceNumber	payrun/summaries/summary/details/detail/invnumber
DetailInfo	decimal TransactionRate	payrun/summaries/summary/details/detail/rate
DetailInfo	decimal Value	payrun/summaries/summary/details/detail/value
DetailInfo	decimal OriginalValue	payrun/summaries/summary/details/detail/origvalue
DetailInfo	decimal BaseValue	payrun/summaries/summary/details/detail/basevalue
DetailInfo	decimal OriginalBaseValue	payrun/summaries/summary/details/detail/origbasevalue
DetailInfo	bool Included	payrun/summaries/summary/details/detail/deleted

the no. 1 in online accounting

The paylist

- Enumerations
- Classes
  - PayOverview
- Methods
  - GetRuns

To be able to process existing runs, it is possible to retrieve a list of existing runs.

Enumerations

```
public enum RunType : int
{
  Payment = 0,
  DirectDebit = 1
}

public enum Phase : int
{
  Select = 1,
  Manipulate = 2,
  Authorise = 3,
  Post = 4,
  Pay = 5,
  Finalise = 6
}

public enum RunStatus : int
{
  Started = 0,
  DoneWithoutErrors = 1,
  DoneWithErrors = 2
}
```

Classes

(d) PayOverview

```
public class PayOverview
{
  public string Office;
  public string PayType;
  public int Id;
  public DateTime Created;
  public DateTime Modified;
  public int Touched;
  public string User;
  public RunType RunType;
  public Phase Phase;
  public PhaseStatus Status;
  public string Bank;
```


the no. 1 in online accounting

```
  public DateTime PayDate;
  public int Year;
  public int Period;
  public bool DirectRemove;
  public string[][] CustomSettings;
}
```

Properties

Property	Description
Office	The office where the pay run belongs to
PayType	The code of the pay type
Id	The unique id of the pay run
Created	The creation date of this run
Modified	The date and time the run data was last modified
Touched	The number of times the run data was modified
User	The user who made the last modifications
RunType	The type of the run
Phase	The current phase of the pay run
Status	The current status of the phase
Bank	The code of the bank that's associated with this pay run
PayDate	The pay date of the payment file
Year	The year where the transactions will be booked on
Period	The period the transactions will be booked on
DirectRemove	The direct Remove setting for this run
CustomSettings	A string array of string arrays with custom pay type settings

Methods

The sessionID must be sent as a SOAP header.

(e) GetRuns

To read all current pay runs from an office, the following function is available.

```
public PayOverview[] GetRuns(
  string office)
```

parameters

Parameter	Description
office	The office from which the current pay runs should be selected

The web service will parse the parameters to an xml document, which should look like this:

```
<read>
  <type>paylist</type>
  <office>string</office>
</read>
```


the no. 1 in online accounting

This xml document will be send to ProcessXML.

ProcessXml will return an xml document that looks like this:

```
<payruns first="1" last="2" total="2">
  <payrun>
    <office name="string" shortname="string">string</office>
    <code name="string" shortname="string">string</code>
    <type>RunType</type>
    <number>int</number>
    <user>string</user>
    <status">RunStatus</status>
    <phase">Phase</phase>
    <paydate>DateTime</paydate>
    <modified>DateTime</modified>
    <touched>int</touched>
  </payrun>
  <payrun>
    <office name="string" shortname="string">string</office>
    <code name="string" shortname="string">string</code>
    <type>RunType</type>
    <number>int</number>
    <user>string</user>
    <status">RunStatus</status>
    <phase">Phase</phase>
    <paydate>DateTime</paydate>
    <modified>DateTime</modified>
    <touched>int</touched>
  </payrun>
</payruns>
```

This xml document will be parsed to an array of PayOverview objects and returned.

the no. 1 in online accounting